Ad Copilot — AI Workflow Design Doc

Status: Draft v1.0 **Owner:** Product, in partnership with ML Engineering **Related docs:** Case Study Narrative · PRD · User Stories · Metrics & Attribution Spec (next) · Architecture Doc (next) · Release Plan (next)

This doc is the technical-product bridge between "we use AI" and "here's exactly how the AI behaves, when it's trusted, and what it costs." It's the document I'd expect to defend line-by-line with an ML engineer in a design review.

---

1. Purpose

Two AI-driven capabilities exist in the product: **creative generation** and **performance recommendation**. Both share a common pipeline shape:

```
Input → Model Call(s) → Confidence Scoring → Lane Assignment → Human Review (if r
```

This doc defines each stage, the model limitations we're designing around, and the cost-control mechanisms baked into the pipeline rather than bolted on after.

2. Model Selection & Routing

Principle: match model cost/capability to the risk and reversibility of the output, not to "use the best model everywhere."

Task	Model tier	Rationale
First-pass creative generation (3-8 candidates per request)	Lower-cost, faster model	High volume, low individual stakes — candidates are filtered before a human ever sees them
Refinement of top-N candidates	Higher-capability model	Low volume (only the winners), higher quality bar since these reach the reviewer
Drift detection &	Higher-capability	Needs to reason over noisy time-series signals and

recommendation reasoning	model, but with strict output schema	produce a defensible written rationale — this is the artifact a marketer will actually read and judge
Brand-safety wordlist screening	Deterministic rule-based check, not a model call	Compliance-critical checks should not depend on probabilistic model behavior; a wordlist/regex layer runs before anything reaches the model-scored path

Known model limitations we design around:

- **Inconsistent scoring across near-identical inputs.** Confidence scores from a single model call are noisy at the individual-item level. We never surface a raw model-generated confidence number directly to the marketer — it's smoothed against a calibration set (§4) and bucketed (High/Medium/Low) rather than shown as a precise percentage that implies false precision.
- **Hallucinated specificity.** Models asked to explain "why" a recommendation was made can invent plausible-sounding but unverifiable causal claims (e.g., citing a competitor action with no data source). The reasoning-generation prompt is constrained to reference only fields present in the input data — no external claims permitted. Output is validated post-hoc against the source metrics before display.
- **Prompt injection via product feed content.** Product titles/descriptions are user-supplied and could contain adversarial text aimed at the generation prompt. Feed content is treated as untrusted input: it's inserted into a clearly delimited data section of the prompt, not concatenated into instruction text, and outputs are still passed through the brand-safety filter regardless of source.

3. Prompt Design Principles

- **Structured input, structured output.** Every prompt requests a JSON-schema-constrained response (variant text + self-reported reasoning tags), not free text, so downstream code can validate rather than regex-parse.
- **Few-shot grounding from the account's own top performers.** Generation prompts are seeded with the account's own historical top-performing ads (not generic industry examples), which measurably improves relevance and reduces generic/off-brand output.
- **Explicit negative constraints.** Prompts state what NOT to do (no unverifiable superlatives like "the best," no pricing claims not present in the feed, no urgency language beyond a configurable tone setting) rather than relying on the model to infer brand safety unaided.

- **Reasoning prompts are separated from generation prompts.** The recommendation-reasoning task and the creative-generation task use different prompt templates and, per §2, different model tiers — conflating them was an early design mistake we corrected, since reasoning-quality requirements are stricter than creative-variety requirements.

4. Confidence Scoring Methodology

Confidence is not the model's raw self-reported score. It's a composite:

```
Confidence = f(model self-score, historical acceptance rate for similar items, da
```

- **Model self-score** — the model's own stated confidence in its output, taken as one weak signal among several, never sufficient alone (see limitation above).
- **Historical acceptance rate** — how often similar past suggestions (same account, same action type) were approved by a human. New accounts have no history, so this term defaults to conservative until enough data accumulates — this is the mechanism that enforces "new accounts can't reach Auto-apply" from the PRD.
- **Data volume** — recommendations based on <7 days or <100 conversions of data are automatically confidence-capped, regardless of how confident the model claims to be, because small-sample noise is a known failure mode in performance marketing specifically.
- **Signal strength** — how far the actual metric is from target, and how stable that deviation has been over the lookback window (a one-hour spike is treated differently than a three-day trend).

This composite score feeds lane assignment (§5) and is recalibrated periodically against actual outcomes (§7).

5. Review Lane Logic (detail)

Lane	Criteria	Example
Auto-apply	Confidence = High AND blast radius below account-configured cap AND account trust tier ≥ established	A 5% budget nudge on a campaign with 60 days of stable data and a strong acceptance history
Suggest	Confidence = Medium, OR blast radius above cap, OR account is newer but has some data	Most v1 traffic lands here by design (per PRD, v1 ships Suggest-only)

Escalate	Confidence = Low, OR data volume insufficient, OR action touches a compliance-sensitive dimension (e.g., new audience targeting)	A recommendation with <100 conversions of underlying data, flagged for a human to decide with full context rather than a lightweight approve/reject

Lane thresholds are configurable per account (US-11) but bounded by hard platform-level ceilings (e.g., no account, however trusted, can enable Auto-apply above a fixed % of daily budget) — this ceiling is not exposed as configurable, to prevent an over-trusting admin from disabling the safety mechanism entirely.

6. Cost Controls (implementation detail)

Building on PRD FR-13–15:

- **Per-account monthly generation cap**, enforced at the request layer before any model call is made (not billed-then-refunded after the fact).
- **Tiered routing** (§2) means the expensive model is only ever called on pre-filtered candidates — rough estimate: if 8 first-pass candidates are generated per request and only top-2 are refined, refinement-tier cost is ~25% of what naive "run everything through the best model" would cost.
- **Caching**: identical product-feed input + no material performance-data change within a rolling window returns cached output rather than a new call. Cache keys include a hash of the feed content plus the prompt template version, so a prompt update correctly invalidates the cache rather than silently serving outdated-logic outputs.
- **Circuit breaker**: if the LLM provider's latency or error rate crosses a threshold, generation requests degrade gracefully (queue for retry, surface a "generation delayed" state) rather than retrying aggressively and compounding cost during a provider incident.

7. Feedback Loop & Model Evaluation Over Time

- Every human decision (approve/edit/reject) on a Suggest or Escalate item is logged and feeds back into the historical-acceptance-rate term in confidence scoring (§4) — this is how the system's trust in itself is meant to actually earn calibration rather than being hand-tuned indefinitely.
- Confidence calibration is reviewed on a recurring cadence against a held-out sample of outcomes (did High-confidence items actually get accepted and perform well?) — if calibration drifts, thresholds are adjusted before considering it a "model problem" to

solve with a bigger model.

- Prompt template changes are versioned and logged against every output (NFR-4 in the PRD), so a regression in acceptance rate can be traced to a specific prompt or model version change rather than treated as unexplained noise.

8. What This Doc Deliberately Does Not Cover

- Model fine-tuning or training — v1 uses off-the-shelf models via API, not a custom-trained model, to keep scope and cost sane for a 0-to-1 build.
- Multi-language support — English-only in v1; prompt templates would need real localization work, not just translation, before expanding.
- Real-time (sub-minute) recommendation latency — hourly refresh is an intentional scope decision, not a technical ceiling; tightening this is a valid v2 conversation once cost and trust dynamics are proven out.